Entrega de Modelagem de Dados

Este documento relaciona cada requisito de Modelagem de Dados ao arquivo do projeto que o comprova.

Período	Responsável
2026	**Equipe do projeto**

Sumário

Requisitos da disciplina e as evidências de cada um

01
Visão geral 3
O que a disciplina pede e como ler o documento

02
Requisitos mínimos 4
Os 7 requisitos para a nota 7,0

03
Requisitos extras 5
Os 7 requisitos além do mínimo

04
Entregas adicionais 6
O que o projeto reúne nesta área

05
Repositórios da disciplina 7
Onde está cada arquivo citado

01

O projeto atende os 7 requisitos mínimos e os 7 extras de modelagem

O que a disciplina pede e como ler este documento?

Disciplina	Requisitos mínimos	Requisitos extras
Modelagem de Dados	**7 atendidos de 7**	**7 atendidos de 7**

Resumo da entrega

A disciplina de Modelagem de Dados pede um banco relacional normalizado, uma carga inicial, a migração de um banco legado, uma camada de BI em modelagem dimensional e transformações em SQL. Os 7 requisitos mínimos correspondem à nota 7,0, e os 7 extras tratam de SQL avançado, otimização, governança e monitoramento.

A estrutura dos bancos principal e de autenticação pertence às migrations Flyway do api-core e do api-auth. O repositório database-console acrescenta, na pasta `db/modelagem`, os schemas, as views, as funções, os triggers e os índices, sem alterar as tabelas do Flyway. Para aplicar esses scripts a um banco, rode `make modelagem-apply` no database-console.

Como ler

Cada requisito aparece com o código (M para mínimo, E para extra), o enunciado, a descrição do que o projeto entregou e até dois links de evidência, que abrem o arquivo na branch main. Quando o mesmo padrão existe em mais de um repositório, o documento mostra um exemplo e cita os outros pelo nome.

A tabela de repositórios, no último capítulo, indica o papel de cada um na disciplina.

02

Os 7 requisitos mínimos estão atendidos e documentados

Como cada requisito mínimo foi atendido?

Código	Requisito	O que o projeto entregou	Evidência
M1	Criar um novo banco relacional PostgreSQL normalizado até a 3FN (ou 5FN, se necessário).	O banco principal é um PostgreSQL com 68 tabelas e 115 chaves estrangeiras, criado por migrations Flyway. A análise de 1FN a 3FN, com as exceções conscientes, está documentada e traz essa contagem.	NORMALIZACAO.md em database-console
M2	Definir PKs, FKs e cardinalidades (`1:1`, `1:N`, `N:N`).	O diagrama entidade-relacionamento do banco principal mostra as cardinalidades, e o SQL declara as chaves primárias, as chaves estrangeiras e as tabelas de relação `N:N`. O mesmo padrão aparece também em api-auth.	POSTGRES.md em api-core V1__baseline.sql em api-core
M3	Carga inicial com pelo menos 500 registros verossímeis.	Um script de carga gera dados verossímeis com a biblioteca Faker. Na cópia restaurada do banco de QA, o `coredb` tinha 4.593 linhas vivas.	dataload.py em database-console RESTORE.md em database-console
M4	Integração RPA para mover dados do banco legado para o novo banco normalizado, garantindo integridade e tratando falhas.	O database-rpa extrai os dados do banco legado e os carrega no banco normalizado de forma idempotente, com a tabela `rpa_id_mapping` para preservar as relações. Um workflow agenda a execução diária.	extended_migration.py em database-rpa scheduled-migration.yml em database-rpa
M5	Camada de BI (Data Mart) com views analíticas em modelagem dimensional (Star Schema ou Snowflake).	O schema `bi` tem as dimensões tempo, fornecedor, demandante e modelo, os fatos de proposta, pedido e cobrança e as views analíticas sobre o esquema estrela. Os scripts foram validados numa cópia de QA.	DATAMART.md em database-console 10_star_schema.sql em database-console
M6	Transformação (ETL em SQL) com CTEs para organizar a lógica.	Todas as dimensões e fatos do esquema estrela são escritos com CTEs (`WITH ... AS`). As views de BI e a de usuários ativos também usam CTEs.	10_star_schema.sql em database-console
M7	Transformação (ETL em SQL) com Window Functions nas views de BI (Running Total, Ranking).	As views de BI usam `SUM() OVER` para o acumulado, `LAG()`, `AVG() OVER` com janela de linhas, `RANK()` e `DENSE_RANK()`. A média móvel de 7 dias dos usuários ativos usa a mesma técnica.	20_bi_views.sql em database-console DATAMART.md em database-console

03

Os 7 requisitos extras cobrem SQL avançado, governança e monitoramento

O que o projeto entregou além do mínimo?

Código	Requisito	O que o projeto entregou	Evidência
E1	SQL avançado, com no mínimo 2 Functions e 2 Procedures para regras de negócio críticas.	O projeto tem as functions `regras.valor_proposta` e `regras.taxa_aceite` e as procedures `regras.recalcular_total_proposta` e `regras.cancelar_propostas_paradas`.	30_procedures.sql em database-console
E2	Triggers de auditoria com NEW, OLD, TG_OP e CURRENT_USER em pelo menos 2 tabelas.	A tabela `audit.log` recebe os registros do trigger `audit.fn_audit`, aplicado às tabelas `purchase_order`, `offer` e `proposal`. Um teste automatizado exige o uso de `TG_OP`, `CURRENT_USER`, `OLD` e `NEW` em ao menos 2 tabelas.	40_audit.sql em database-console test_modelagem.py em database-console
E3	Otimização com análise de consultas lentas por EXPLAIN ANALYZE e aplicação de índices.	A análise foi feita numa cópia de QA com massa sintética. A consulta por situação e período caiu de 62,1 ms para 3,7 ms com o índice `idx_proposal_status_created_at`.	OTIMIZACAO.md em database-console 80_indexes.sql em database-console
E4	Governança com catálogo de dados (tabela técnica de metadados com tabelas, colunas, regras de negócio e níveis de acesso).	A tabela `governance.data_catalog` guarda as regras por tabela e é atualizada pela procedure `refresh_data_catalog`. O contrato de acesso por consumidor, com os papéis de leitura, fica em `contracts.json`.	50_data_catalog.sql em database-console
E5	Governança com procedimentos documentados de backup e recuperação de falhas.	O fluxo de backup do PostgreSQL e a restauração passo a passo estão documentados, com a conferência de tabelas, migrations e linhas.	BACKUP.md em database-console RESTORE.md em database-console
E6	Monitoramento com registro automatizado de acessos DAU (Daily Active User) diretamente no Postgres.	A tabela `monitoring.daily_active_user` é preenchida por um trigger em `auth_session`, e a view `vw_dau` consulta os usuários ativos por dia.	60_dau.sql em database-console
E7	Estruturas avançadas, com CTE recursiva ou herança de tabelas, onde fizer sentido.	A view `bi.vw_rede_conexoes` usa uma CTE recursiva para percorrer a rede de conexões até 3 graus, sem ciclos.	70_recursiva.sql em database-console

04
Entregas adicionais

O que o projeto reúne nesta disciplina?

O projeto mantém um seed determinístico que povoa os bancos de negócio e de autenticação, o acervo mocks-warehouse, que fornece as imagens referenciadas pelas entidades dessa carga, views de exportação no schema export do banco de negócio, papéis de leitura por consumidor com privilégios declarados em contrato e um schema de indicadores com segurança por linha. Esses recursos ficam em database-console, api-core e mocks-warehouse.

Link de exemplo, contracts.json em database-console.

05

Cinco repositórios guardam o banco, a carga, a migração e o Data Mart

Em qual repositório está cada arquivo citado?

Repositório	Papel na disciplina	Link
api-core	Banco principal, com as migrations Flyway, o diagrama do banco e as views de exportação.	Abrir o repositório
api-auth	Banco de autenticação, com as próprias migrations Flyway e o diagrama.	Abrir o repositório
database-console	Normalização, carga inicial, Data Mart, SQL avançado, otimização, governança, monitoramento e backup.	Abrir o repositório
database-rpa	Integração RPA que move os dados do banco legado para o banco normalizado.	Abrir o repositório
mocks-warehouse	Acervo de imagens referenciadas pelas entidades da carga de dados.	Abrir o repositório



Entrega acadêmica.